HLTP/x25-32 Full Microarchitecture and Instruction Set Architecture Specifications

CPU Rev. 0.3-indev
Document Version 0.1

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

Contents

Introduction

Chapter	*Title*	*Page(s)*
0.	**Preamble**	3
0.1	Technical Terminology, Abbreviations, Typographic Conventions	4

Processor Architecture

Chapter	*Title*	*Page(s)*
1.	**Abstract Design**	5
1.1	Common CPU Specifications	6
2.	**Logic Hardware**	7
2.1	General Processor Layout	
2.2	Busses	
2.3	Registers	
2.4	Processing Internals & Internal Registers	
2.4.1	Control Unit	
2.4.2	Arithmetic Logic Unit	
2.4.3	Memory Management Unit	
2.5	Hardware Memory Layout	
2.5.1	MMIO	

Runtime Modeling

Chapter	*Title*	*Page(s)*
3.	**Instruction Set Architecture**	
3.1	Instruction Encoding	
3.2.1	Instructions Master Table	
3.2.2	List of Instructions	
3.3	Conditionals	
3.4.1	Assembly Syntax	
3.4.2	Special Assembly Constructs	
4.	**Execution**	
4.1	Fetch-Decode-Execute Cycle	
4.2	Reset / Boot Process	
4.3	Interrupts	

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

0: Preamble

Disclaimer: This project and all related content are **not** intended for any real world application. It is primarily intended as my (the author's) way of learning CPU design, aimed at providing educational use of computer architecture theory. Although trying to imitate real world implementations, some areas may significantly deviate from standard use/understanding; This usually comes down to either intentional design decisions or incompetence (by the author of this document) to fully comprehend architecture design.

Before you read this document, please be aware that it is assumed, that the reader has already familiarized with some basic computer knowledge and additionally ground level knowledge of low level CPU architecture design.

This document is structured in a way, such that information can be quickly looked up if necessary, while still retaining a somewhat in-depth explanation of each detail.
Most of the information will be written, such that this document best explains not only how the processor interacts with peripheral media, but more importantly what happens inside. The goal is to make you, the reader understand what is happening and why it is that way.

Some design decisions have been arbitrarily made, some are by convention, most are against convention, and some decisions I regret from not having learned enough early enough.
If you want to improve on anything, try it for yourself, since ultimately this is just another learning tool instead of some real-world applied high-performance cutting-edge technology shenanigans.

Note: This documentation is essentially split into two pieces.
One is this, the actual hardware and ISA documentation, while the other "auxiliary documentation" provides more info on the circumstances of this project, how it came alive, and most importantly, a list of useful (internet) resources attached.

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

0.1: Technical Terminology, Abbreviations, Typographic Conventions

CPU	Central processing unit
CPI	Clocks per instruction (Multiplicative inverse of Instructions per clock / IPC)
ISA	Instruction set architecture
MSB	Most significant byte
LSB	Least significant byte
ISR	Interrupt service routine
IO	Input / output
MMIO	Memory-mapped IO
FDX (Cycle)	**F**etch-**D**ecode-e**X**ecute Cycle

Tab. 1.1: Commonly used abbreviations throughout this document

GPR / GP[X]	General-purpose register [X]
PC	Program counter (commonly also referred to as Instruction Pointer / IP)
IR / IRA / IRB	Instruction register / Instruction register A / Instruction register B
MAR	Memory address register
MDR	Memory data register
SR	Status register
CU	Control unit
ALU	Arithmetic logic unit
MMU	Memory management unit
RAM	Random access memory (sometimes just called “Memory”)
CON / TMEM	Console memory
VMEM	Video memory / Graphics memory

Tab. 1.2: Abbreviations for CPU hardware internals

[TODO: Possibly refer to internet links (e.g. OSDev wiki)?]

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

0.1: Technical Terminology, Abbreviations, Typographic Conventions

Typographic conventions:

bold:
denotes special, important meaning in the given context

bold & italic:
references to special terminology (from Tab. 1.1 & 1.2)
(only if relevant to the context)

italic:
cross-references, special / undefined values

`monospace:`
part of source code, file names, or shell commands

<inside angle brackets>:
denotes a value to be replaced by what the user desires. A description of permissible values is usually provided.

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

Chapter 1:
Abstract Design

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

1.1: Common CPU specifications

Word Size	32 bits / 4 bytes
Instruction Width	64 bits / 8 bytes (double word alignment)
Endianness	Big-endian (MSB first, LSB last)
Registers	8x GPRs (1 word), PC (1 word), IRA (1 word), IRB (1 word), SR (1 word), SP (1 word), MAR (1 word), MDR (1 word)
Clocks per instruction (CPI)	*- in revision -*
Interrupts	*- in revision -*
Architecture model	RISC
Pipelining	None
Branch prediction	None (requires Pipelining)
Parallelism	None
Cache architecture	No cache
Virtual memory support	Hardware address mapping, No dynamic remapping during execution
Execution modes	Real / direct mode only, No mode-switching instructions
Extensibility	Not implemented (No co-processing)

Tab. 2: Specifications common among different CPU architectures

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

Chapter 2:
Logic Hardware

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

2.1: General Processor Layout

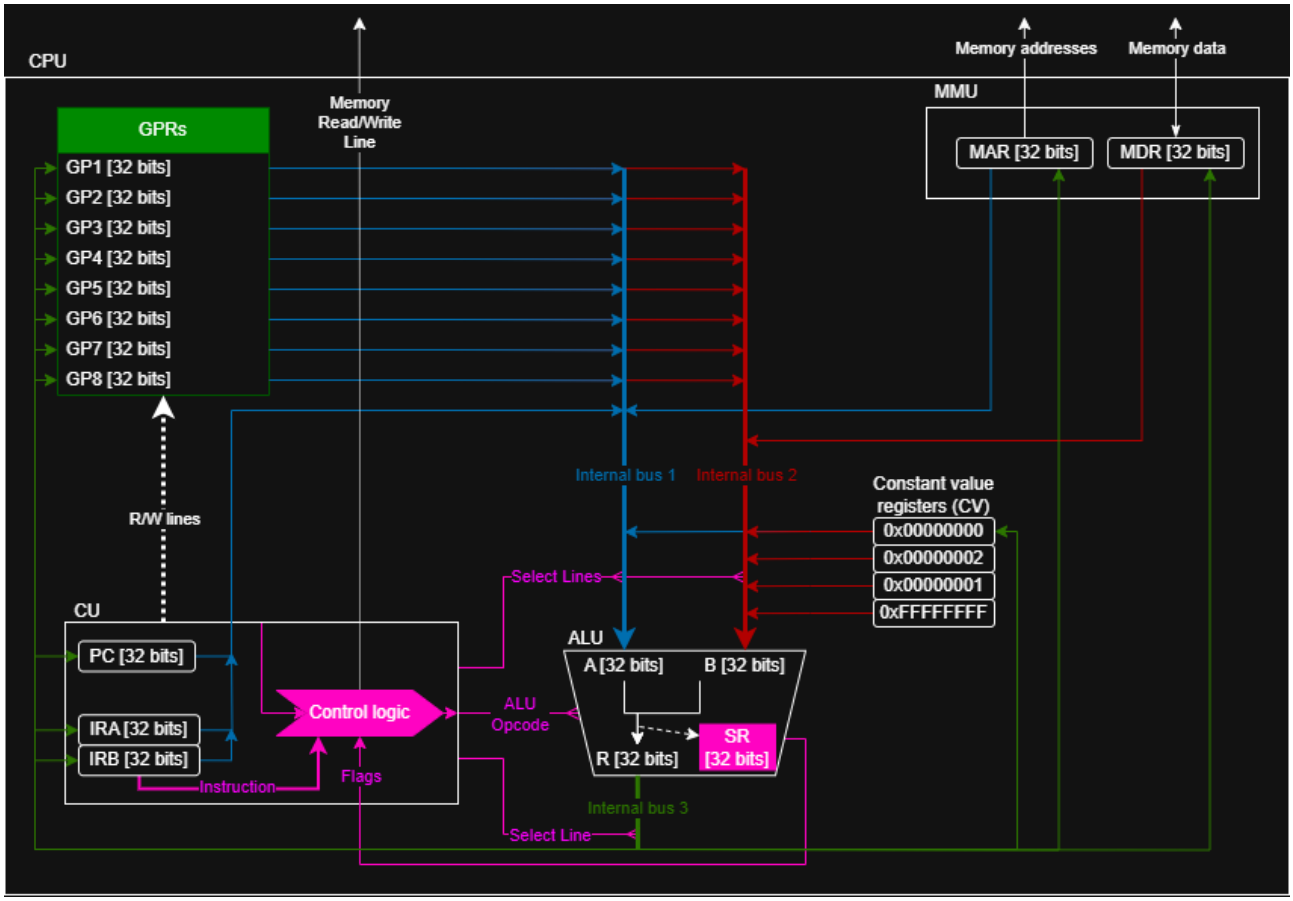

Fig. 1: The x25-32 CPU Hardware Layout as of Processor Revision 0.1-indev

Legend:
Component colors:
Pink: Elements directly affecting, processing or forwarding control signals
Blue: Direct attachments to Internal bus 1
Red: Direct attachments to Internal bus 2
Green: Direct attachments to Internal bus 3
Lines:
Dotted: Single bit line / single “wire”
Solid: Bus (32 bits)
Solid and fat: Bus (32 bits) that changes source / destination depending on control signals (using multiplexers); *Exception: “Instruction” bus from IRA + IRB to Control logic. This line is only fat to emphasize that it carries 2 words from IRA and IRB at the same time, meaning it is 64 bits wide.*
Miscellaneous:
Rounded boxes: Special purpose registers

Note: The constant value registers cannot be written to (writes are essentially ignored). The reason the ***zero*** register (0x00000000) still has an input is that it acts as a “discard” register; When the ALU’s operation has no target, this register is therefore the destination of Internal Bus 3, where the ***zero*** register is also referred to as the ***void*** register.

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

2.3: Registers

Name	Size	Location (logical)	Function / Purpose	Special notes
General-Purpose Register X[1] ***GPX***	32 bits (1 word)	***CPU global***	A general purpose fast temporary storage register used for immediate calculations.	Not to be used by the hardware, but exclusively freely by the software.
Program Counter ***PC***	32 bits (1 word)	***CU***	Stores the address of the next instruction	Incremented BEFORE an instruction is executed → This ensures the instruction itself (e.g. JMP) may override the PC without added complexity afterwards
Instruction Register A ***IRA***	32 bits (1 word)	***CU***	Stores the first word of the current instruction	-
Instruction Register B ***IRB***	32 bits (1 word)	***CU***	Stores the second word of the current instruction	-
Status Registers ***SR***	32 bits (1 word)	***ALU***	Each bit of the SR represents a “flag”, which is either 1 to represent a state is present or 0 to represent it is absent.	-
Stack Pointer ***SP***	32 bits (1 word)	***MMU***	Stores the memory address of the uppermost stack element.	-
Memory Address Register ***MAR***	32 bits (1 word)	***MMU***	Stores the memory address the CPU wants access to.	-
Memory Data Register ***MDR***	32 bits (1 word)	***MMU***	Stores either… 1. ...the value at the address in the MAR. 2. ...a value to be stored at the address in the MAR.	Acts as both an input and an output register depending on the memory R/W line.

Tab. 2.2: Summary of all CPU registers

[1]: The entry applies to every GPR (GP1 through GP8), so GPX is used as a placeholder.

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

2.5: Hardware Memory Layout

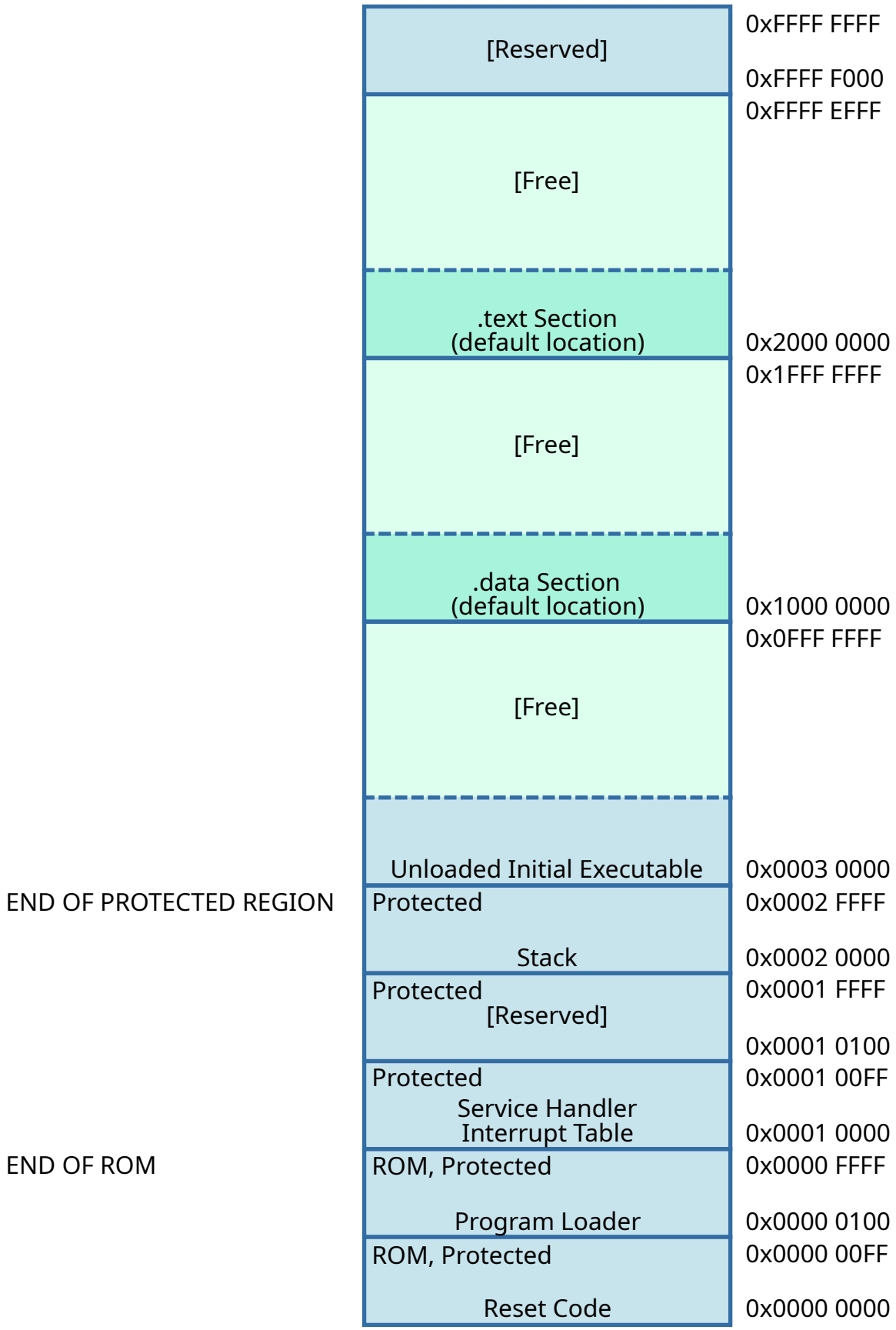

Fig. 2: The x25-32 Hardware Memory Layout as of Processor Revision 0.2-indev

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

- Important: This data is mostly obsolete and currently being reworked -

This section only describes the *hardware enforced* memory layout.
The address space of the CPU is required by the CPU itself to have this structure, since it otherwise results in undefined behaviour.
Any further software-driven subdivision is possible at the discretion of the user.
The described memory layout is outlined in Fig. 2.

Legend:
Memory sections marked as ROM are on read-only devices.
Memory sections in the protected region trigger an *illegal secure memory write exception* when written to from an unprivileged point (more in about this later in this chapter).
Green sections are usually free to use by the main program, which is loaded by the program loader.

The following table explains every region in more detail:

Region name	**Explanation**
Reset Code	The reset code is a directly executable part of memory (starting at 0x00000000), which immediately runs once the CPU has experienced a hardware reset. This can either be caused by a software interrupt to the reset vector or a general hardware / software fault triggering an unrecoverable condition. The reset code ensures that the CPU launches in a “clean” state: 1. It clears all GPRs to zero, 2. resets the Stack Pointer to its base, 3. clears the Status Register, 4. and finally calls the loader.
Program Loader	The program loader is executed once the CPU reset code has finished running. It is responsible for loading an unloaded executable from address 0x00030000 onward into its proper location to be executed: 1. It copies the unloaded executable into the load address it specifies, 2. calculates its entry point, 3. and finally jumps to the entry point, leaving execution to the user program.
Service Handler Interrupt Table (SHIT)	The Service Handler Interrupt Table (in short: SHIT) contains information about where each interrupt vector resides in actual address space. Each offset into the SHIT (interrupt vector) has a memory address, which points to the actual ISR.
[Reserved]	This memory region is reserved for later use. Do not use.
Stack	The CPU global stack is FILO (first-in-last-out) memory structure, which allows for storing temporary data (most commonly part of the CPU state) and later retrieving it. Instructions that interact with the stack are most notably “push” and “pop”, but also any instruction that use a subroutine, which is able to return via either “ret” or “iret”. Those instructions include “call” and “int”.
Unloaded Initial Executable	The unloaded initial executable is a binary copy of what the assembler produces when assembling from assembly code. It

	contains header information, a .data and a .text section without properly aligned addresses, so it won't run directly when called (since it expects "wrong" offsets in memory). The aforementioned Program Loader is responsible for loading this program into memory and executing it.
.data and .text section default locations	*-in revision-*
[Reserved]	This memory region is reserved for later use. Do not use.

Tab. 3: Explanations for each memory region

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

2.5.1: MMIO

MMIO is to be implemented and not part of CPU revision 0.3-indev.

Memory-mapped input/output
- Video buffer x2 (inactive, active; RGB)
- Text only (output)
- Text input buffer (keyboard, serial)
- Interrupt data (also keyboard, but others too)

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

Chapter 3:
Instruction Set Architecture

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

3.1: Instruction Encoding

Encoding Overview:
- Types (R/I/J)
- Encoding in binary of all three
- Example

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

3.2.1: Instructions Master Table

Table for general quick overview:
- Opcode, mnemonic, (usage) format

3.2.2: List of Instructions

For every instruction (not in a table):
Mnemonic
Format
Operation
Type
Opcode
Parameters
Encoding (e.g. `0x63af#### 0x########`)
Flags affected
Exceptions / interrupts / “traps”

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

3.3: Conditionals

3.4.1: Assembly Syntax

To distinguish between the program that is used to assemble a given assembly program and the assembly program itself, the following terminology is used:
- “Assembly (code)” refers to the assembly code, i.e. the source file’s human-readable content itself
- “Assembler” refers to the program responsible for converting “assembly” code into a machine-readable binary format.
- “Machine Code” is the executable binary sequence from an assembler and the result of assembling an assembly program.

A HLTP32 assembly source file in general has the following code structure:

```
#VERSION 3.1
#ORG <loadaddress>

<list of pre-processor directives>

#SECTION <data attributes> .data
<list of data>

#SECTION <text attributes> .text
<list of instructions>
```

Every line in the assembly code is referred to as a “**statement**”.

Licensed under the GNU GPL v3, Redistribution allowed under same license only.

Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045

Directives:
Every statement starting with a “#” signifies a **pre-processor directive** (or **directive** in short).
Additional to that, there also exists the `times` keyword, which is also a directive.
A directive generally signifies something to the assembler without directly being part of the resulting binary machine code.
The following pre-parser directives exist:

Directive	Description
`#VERSION`	Exists since the assembler version 2.0 to signify the version of the assembler used, to distinguish both syntactical, as well as CPU revision specific behavior. If this version and the assembler’s version mismatch, an error should be raised, if the assembler is not written such that is handles older assembly code as well. Must be the first line in the assembly code.
`#ORG`	The address after this directive is the global absolute load address of the program, i.e. the address at which this program is expected to be loaded at to properly work. Must be the second line in the assembly code.
`#DEFINE <ALIAS> <VALUE>`	Replaces every occurrence of `$<ALIAS>` in the code following this directive with `<VALUE>`
`times <N> <STATEMENT>`	Repeats the `<STATEMENT>` following the directive `<N>` times.

Tab. 3.4.1.1: A list of all assembly pre-parser directives and their use

General layout
- #VERSION, #ORG
- pre-parser (#define, times N)
- .data and .text sections

.data:
[label] dw [data]
or
equ [name] [value]
.text:
[label]
[mnemonic] [op1], [op2]

Comments:
; ignored

In-code arithmetic
Padding of strings, multi-words, and integers (depending on fmt 0x, 0b, or dec)

Conventions:

- special assembly labels are prepended with an underscore. What is regarded special is up to the user. Examples: _start, _halt

Result: Header, Data, Text

3.4.2: Special Assembly Constructs

4.3: Interrupts:

SHIT

Interrupts are just special subroutines:
- They affect interrupt flags
- The instruction operand is not an address, but a vector (requires indirection via SHIT)

When interrupt is caused:
1. push SR to stack
2. "Interrupt enable" flag is cleared (set to 0)
3. "Interrupted" flag is set
4. push PC to stack
5. Extract interrupt vector from instruction operand, get address from vector, jump to vector address
6. ... ISR code runs ...
7. ISR eventually ends with the "iret" instruction
8. Pop PC from stack
9. Pop SR from stack
10. Resume execution at PC

→ Also check for double fault (when interrupted flag is already set before ISR start)

Hardware IRQs?

Licensed under the GNU GPL v3, Redistribution allowed under same license only.
Part of the TinyProc Project (2025 - 2026) by nick16384 and Maxi12045